Addressing imPAIRED observations

Charlie Gauthier
Université de Montréal
charlie.gauthier@umontreal.ca

David Dobre
Université de Montréal
david.dobre@umontreal.ca

Adam Sigal
McGill University
adam.sigal@mail.mcgill.ca

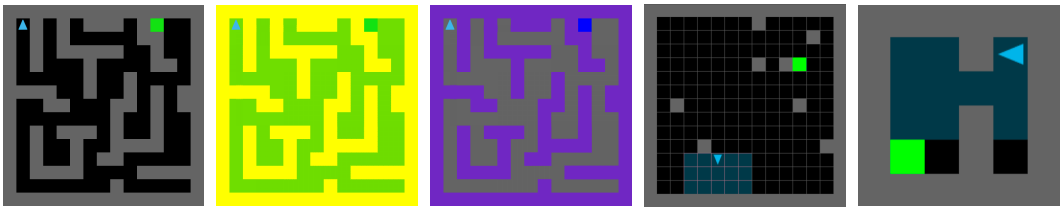
Abstract

A major problem in robotics is that models trained in simulation do not transfer well to real-world settings. Some approaches exist to make this transition more successful, such as Active Domain Randomization (ADR) where a model learns to change the domain in which the main RL agent's environment, generating a series of increasingly difficult domain shifts. This technique is related to Curriculum Learning (CL), wherein agents are trained on progressively more complicated tasks in order to "flatten" their learning curve, enabling them adapt to new, more complex environments efficiently. PAIRED by Dennis et al. [2020] is a recent advancement in CL which allows curricula to be generated automatically. PAIRED works in a similar way to ADR in that it generates tasks that are *just* outside of the agent's current capabilities. In this work, we combine both approaches. We hypothesized that by allowing the adversarial environment designer to alter the representation (i.e. colour) of each block type as observed by the RL agent, it would achieve greater robustness and generalizability. However, simply changing the observation scheme of the RL agent from three fixed *block types* to a larger set of distinct colours, led to an unexpected diminishing of performance. In fact, the essential results of Dennis et al. [2020] itself were unable to be reproduced under this new observation scheme. This demonstrates a crucial failure case of the original work, and indicates the need for further research into adversarial control of observation domain. Nonetheless, our results reflect higher robustness for agents trained in an adversarial observation domain versus the PAIRED baseline, illustrating the potential of this new training paradigm.

1 Background

Reinforcement Learning (RL) is a machine learning paradigm in which a learning agent seeks to learn a policy that can maximize a notion of cumulative reward inside of a given environment. It is a powerful framework because it enables end-to-end learning of an agent's behaviour. However, it suffers from many problems such as sample inefficiency, training instability, and agent generalization to new domains. A popular technique used in the context of RL to mitigate some of these issues is Curriculum Learning (CL). This approach trains RL agents on a "curriculum" of logically ordered, progressively harder tasks, which has been empirically shown in Bengio et al. [2009] to improve both the convergence time and quality of local minimum obtained, enabling agents to generalize better.

A recent paper by Dennis et al. [2020] proposed a new multi-agent RL setup for CL which they call "Protagonist Antagonist Induced Regret Environment Design" (PAIRED). This work uses three agents: one adversarial agent that generates environments, and two agents that train in that environment. The goal is for the adversarial agent to generate environments that are easy for one agent (the antagonist) to solve, but difficult for the other agent (the protagonist). This is accomplished by having the antagonist pre-train to near-optimality before training the protagonist, and then using the *regret* (approximated as the difference between antagonist and protagonist payoffs) to train the three agents.



(a) (b) (c) (d) (e)

Figure 1: Among (a, b, c), PAIRED's agent can only really solve grid (a), which is the training domain. Humans can easily solve all (a, b, c) because we rely on patterns and high level features more than colors. (d) is a sample environment generated by our PAIRED baseline adversary after 400k timesteps. (e) is a sample from `MultiGrid-MiniTwoRooms-Minigrid-v0`, a 7×7 grid that we evaluate on.

The protagonist is optimized to minimize regret, while the antagonist and adversarial agents are optimized to maximize the regret.

Even with PAIRED or other CL methods, RL agents are very challenging to train. Many rollouts into an environment are required to collect enough feedback to change the policy. This problem is particularly egregious in the field of robotics, where in addition to the complications related to resetting the environment after each rollout, training RL with real robots is fraught with safety concerns. This is why we train robotic RL agents in simulators: being able to reset and reproduce environments is a great boon to training. But simulators are clearly never perfect, especially considering how information-dense the real world is. The dynamics, colours, and even reward signals are often inaccurate. This means that agents trained in simulation often fail to transfer to the real world. There exists a large literature on that "sim-to-real" task (see James et al. [2018]).

One technique that aims at solving this task is Domain Randomization (Tobin et al. [2017]), where the environment's domain is randomized (while keeping its task static) with the goal of improving the agent's generalization prowess. Active Domain Randomization (ADR) (Mehta et al. [2019]) tries to generate domain configurations that are *just* outside the agent's current capabilities in order to create a curriculum of domain shifts. The issue is that these methods are often very inefficient, and lead to high variance in the trained agents Mehta et al. [2019].

PAIRED and ADR are quite similar in their approach. PAIRED changes the environment (e.g. map configuration) that the agent has to solve, while ADR keeps the general environment the same but changes its domain (dynamics, observations, probability transition functions, etc). The goal of this work is to combine ADR with PAIRED such that the adversarial environment agent incorporates observational domain exploration in the generated curricula.

Note that in PAIRED, the authors use the term "domain shift" to only refer to changing the environment configuration. While technically correct, there is a clear difference between changing how many things are inside the environment and where they are located, and changing how the things inside the environment look. The latter is more in line with robotics research and sim2real, where being robust to observational changes is a critical challenge to overcome.

To motivate our work, consider the example grid-world environments in Figure 1. It is trivial for humans to disentangle the task from the specific observable representations, and understand which new colour represents which type of block and how to correctly interact with them to reach the goal. However, RL agents trained without observational domain changes during training tend to only be able to solve tasks in their own source domain (see James et al. [2018]). This is true for PAIRED. As we will demonstrate in our results section, protagonists trained by PAIRED tend not to generalize well when the observations of the environment are shifted (even if the shift preserves all data that is relevant to solving the task). To mitigate this issue, we will extend the action space of PAIRED's adversarial agent (which generates environments) to have the ability to control the observational representation of the environment. In the context of a grid-world, a domain shift refers to changing the colours of the objects in the grid. We call our extension *imPAIRED*. This approach allows us to remain in the curriculum learning regime by having the adversarial agent learn how to generate a progression of domain shifts that would most optimally train the main agent.

2 Approach

In this section, we will briefly discuss the core idea behind PAIRED and then detail our modification to it to support domain shifts.

2.1 PAIRED

To provide a useful learning signal to the adversarial environment agent, PAIRED makes use of minimax regret based on work by Savage [1951]. This formalizes the difference between the reward received for taking an optimal action, and the reward received for an agent's chosen action. The formula for regret in PAIRED (and our imPAIRED extension) is:

$$\text{REGRET}^{\overrightarrow{\theta}}(\pi^P, \pi^A) = U^{\overrightarrow{\theta}}(\pi^A) - U^{\overrightarrow{\theta}}(\pi^P), \tag{1}$$

where $\overrightarrow{\theta}$ is the set of parameters describing generated environment, π^P is the protagonist's policy, π^A is the antagonist's policy, $U^{\overrightarrow{\theta}}(\pi^P)$ is the reward obtained by protagonist, and $U^{\overrightarrow{\theta}}(\pi^A)$ is the reward obtained by antagonist. The antagonist's reward acts as a proxy for optimal behaviour, making equation (1) reasonably model the extent to which the protagonist has missed potential in its actions. Under this scheme, the adversarial environment aims to maximize the antagonist's reward when creating environments. The antagonist's objective is to *maximize* regret (i.e. aim for its own reward to be high, while keeping the protagonist's reward as low as possible), while protagonist's objective is the opposite, aiming to minimize regret. The way regret is approximated in practice is to evaluate multiple instances of both the protagonist and antagonist in the generated environment, and then compute the difference between the maximum reward obtained by the antagonists and the mean reward obtained by the protagonists.

Dennis et al. show that this setup incentivises the adversary to create environments which are easy for the antagonist to solve, but difficult for the protagonist. Using the two RL agents in this way allows for task generation that is *just* outside of the protagonist's current abilities, without being so difficult as to impede learning, or being completely impossible. This last point is crucial. Other approaches, such as minimax adversarial training (which only uses a single learning agent with an adversarial environment agent), converge to situations where the environment generates configurations that are impossible to solve such as blocking goals within a room with no doors.

2.2 imPAIRED

It is simple to extend the framework of PAIRED to include a domain shift action. When generating environments, the adversary's action at every step is to select the location of the of where the next object is placed. The first object is the agent, the next object is the goal, and the remaining objects are walls. To handle collisions, if the goal is placed on the agent, the goal is randomly placed on the grid. If a wall is placed on any other wall, no object is placed. We can add another three steps at the beginning of the adversary's episode which correspond to selecting three colours. This process is then repeated for 50 steps. In this way, the adversary selects:

```
WALL_COLOR, GOAL_COLOR, FLOOR_COLOR, AGENT_LOC, GOAL_LOC, WALL1, WALL2...
```

This way, the parameters describing the generation of the observation space are contained in $\overrightarrow{\theta}$, like the parameters describing the generation of the environment already are. We simply increased PAIRED's environment generation capability. From PAIRED's publication, we know that this should converge to good antagonist, protagonist, and adversary policies.

We refer to the action of changing of the observation space as the *domain shift action*. We hypothesize that agents trained in adversarial environments with this action enabled will learn to identify patterns in their observations, rather than relying on knowing the exact nature of each type of block (if the adversary learns to select a diverse set of domain shifts during the protagonist's training).

Note that when generating environments, the adversary also uses its own observations in its policy to select the next action when generating an environment. This means that the adversary is going to be changing the domain of its own observations too, which complicates its task.

3 Related Work

An earlier example of similar work was conducted by Sukhbaatar et al. [2017], where the authors introduce a method called *asymmetric self-play* (ASP). Their process involves pitting two versions of the same agent against one another, whom they nickname "Alice" and "Bob". Alice proposes a task for Bob to complete, and then Bob attempts to do so. The authors crafted a reward structure that rewards Alice for proposing tasks that push Bob ever-so-slightly out of his comfort zone, while not being too difficult for him to complete. This regime allows for automatic curriculum generation of increasingly difficult tasks. Asymmetric self-play was found to reduce the number of *supervised* training steps required, and in some cases converges to a higher reward.

The ASP concept is extended in Sukhbaatar et al. [2018], with a method focused on learning goal embeddings via self-play for hierarchical reinforcement learning. The method consists of dividing the goal-setting task between high and low-level policies. To do this, a third agent, "Charlie", was added, which takes charge of high-level policies, generating continuous sequences of sub-goals. A lower-level agent (Alice) then learns to efficiently represent the sub-goals, and executes them. Finally, as in ASP, Bob learns by reproducing Alice's proposed tasks. The work found that this process generates a logical decomposition of tasks within the goal embeddings, and overall, significant performance gains over non-hierarchical approaches. Our proposed work also introduces a high-level agent, however it is tasked with changing the environment itself, rather than sub-goal generation.

This approach is very similar in concept to our initial plan for the study, wherein we planned on having an adversarial agent create environments for a protagonist agent. After finding Dennis et al.'s study, we decided to also incorporate the antagonist into our experiment, as they proposed, in order to prevent the adversary from proposing unfeasible tasks.

4 Method

Dennis et al. [2020] made PAIRED's code available freely on GitHub[1]. We used their code as a baseline, and extend it to support our experiments. Due to resource limitations, we constrain our training and evaluation experiments to a simplified and smaller scale than what Dennis et al. [2020] used. Specifically, we train fewer agents in parallel (4 instead of 30) but for the same number of steps (500k).This ultimately results in poorer agent performance, which we will account for by evaluating the agent in simpler, smaller environments.

In the original work by Dennis et al. [2020], the objective was to show that an agent trained with the PAIRED framework generalizes to unseen environment configurations in a zero-shot manner. For example, having only trained on environments produced by the adversarial environment, they would then test the agent in a variety of predefined environments such as a maze, a multi-room environment, or a randomly cluttered environment. The objective in these environments is for the agent to find and navigate to a goal block, where the observations that the agent receives are limited to a smaller region in front of the agent rather than the global map. This "imperfect information" regime is a much harder task in the context of RL (Jaakkola et al. [1999]), and requires the agent to search for the goal.

Upon successfully completing a task, the environment returns a reward:

$$R = 1 - 0.9 \times \frac{\text{steps taken}}{\text{max steps}} \tag{2}$$

where steps taken is the number of steps the agent took before arriving the the goal, and max steps is the maximum number of steps the agent could take before the environment terminated and returned zero reward. This is a sparse reward signal, as no other reward signal is offered by the environment. Note that this reward does not necessarily have ideal properties in extreme cases: consider a scenario in which the agent and goal are on opposite sides of the grid, with a mess of walls to navigate through. Even if the agent takes the optimal path, it is still penalized by the number of steps it had to take despite potentially perfectly navigating a very difficult environment. A potentially better metric would be to use a pathfinding algorithm to find the shortest path, and scale the number of steps by the number of steps in this shortest path. We do not explore this however, and leave this for future work.

At a high level, we run the same tasks, but we also test these environments with different domain shifts (i.e. different colours) to see how robust an agent is to these shifts. In order to ensure our results

[1] https://github.com/google-research/google-research/tree/master/social_rl

are as directly comparable as possible, we used the same grid world simulator their code used. This means that if our framework works, we can view our simplified experiments as "preliminary results", and then trivially re-run them with the same complexity that was used in PAIRED to obtain a more faithful comparison.

4.1 Implementation

PAIRED extends the gym-minigrid environment (Chevalier-Boisvert et al. [2018]) to implement the adversarial environment-generating agent. The original gym-minigrid uses a unique representation for the observations. It is a matrix where each entry is a tuple of the form `(BLOCK_TYPE, BLOCK_COLOUR, STATE)` where `BLOCK_TYPE` is an integer encoding of the object type (e.g. "wall"$\rightarrow$ 2, "floor"$\rightarrow$ 3, etc.), `BLOCK_COLOR` is an integer encoding of the discrete colour string (e.g. "red"$\rightarrow$ 0, "green"$\rightarrow$ 1, etc.), and `STATE` is the state of the object, if relevant (e.g. a door can be open or closed, but a wall has no state). As mentioned above, we restrict ourselves to a simpler class of grid-worlds without any interactive objects where `STATE` is always 0 (such as multi-rooms or cluttered environments).

The fact that the block type is embedded directly in the representation of each block is clearly an issue for our experiments. Even if we changed the `BLOCK_COLOUR` value, the agent could still trivially learn to slice the matrix down to its first channel, selecting only the block's type, and then learn a policy on that slice. To account for this, we change the encoding of the representation: `(BLOCK_COLOUR, BLOCK_COLOUR, STATE)` Instead of defining blocks by their type directly, we use only their colour. An example comparison between the original representation and our new encodings is shown in Table 1. During the adversary's domain shift action steps, if it picks an already selected colour, it instead picks the next available colour, and so each colour is only used for one type of block each, per domain shift. We don't lose any information here: in essence, the colour is really the block's type too! Note that one caveat in our new representation is that we effectively remove a dimension from the representation. We speculate that this could be source of error in our experiments; this is discussed further in Section 5.2.

5 Experiments and Results

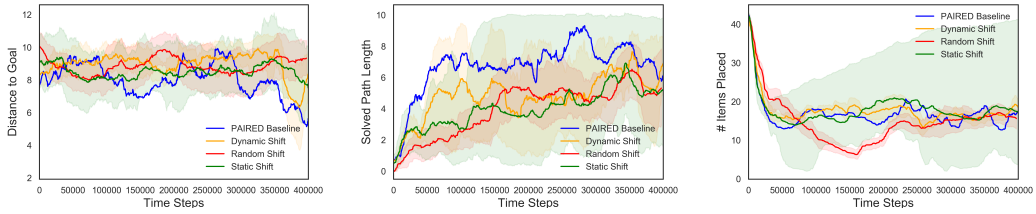
We train a number of different agents under a number of different regimes to test our hypothesis that the inclusion of a domain shift in the adversary's action space would result in agents that are more robust to domain shifts. In Section 5.1, we describe how we train these agents. In Section 5.2, we discuss the training metrics we obtain and any other observations we have throughout the training process. In Section 5.3 we discuss how these agents perform on different tasks and different domain shifts, both seen and unseen.

5.1 Agents

In order to test our hypothesis, we trained the following agents:

1. An agent is trained with the adversary's domain shift action disabled. This is referred to as the **source agent**, and its training environment is the **source environment**.
2. We find an environment with altered block type representations such that the agent no longer knows how to behave successfully. This is the **target environment**. This is done to ensure that a simple shift of observations is enough to break the agent. This is used as our baseline to characterize how badly an agent trained without the domain shift action does when transferred to a new domain-shifted environment.
3. With the domain shift action enabled, a second agent, the **transfer agent**, is trained. It is expected to generalize better to the target environment than the source agent.
4. To evaluate whether or not agents trained with domain randomization perform worse than those trained with imPAIRED, we train a third **random agent** using domain randomization. During the training of this agent, the observation space will be repeatedly randomly altered (domain shifts). The randomness of this agent refers to its training on random domain shifts, rather than its policy being random.

Given the above experiments and our hypothesis, we expect the following results:



(a) EMA of Distance to Goal (b) EMA of Solved Path Length (c) EMA of Number of Items Placed

Figure 2: Performance of agents trained on PAIRED baseline, Dynamic Shift, Random Shift, and Static Shift. Error bars are the min and max of four different runs. Note that we only trained one PAIRED baseline agent (blue), so this has no uncertainty estimate.

1. In its **source environment**, the **source agent** will demonstrate high performance.
2. The **source agent** will perform poorly in the **target environment**(s). This is trivially logical because the target domain is expressly chosen to be difficult for the source agent.
3. The **transfer agent** will have similar performance on both the **source environment** and the **target environment**(s). Its performance on the **target environment**(s) will be better than that of the **source agent**.
4. The **random agent** will perform more poorly than the **source agent** on the **source environment**, but better than the source agent on the **target environment**.

In addition to the above, because pretrained weights are not published, we attempt to reproduce the results obtained by Dennis et al. [2020] by rerunning their unmodified code with the original minigrid environment (and encodings). This is done so that we can verify that our encoding modifications to the environment were bug-free: we compared the source agent evaluated on our encodings, to their baseline evaluated on their encodings to ensure that there were no significant behavioural difference. We also evaluate the baseline PAIRED agent (trained on the original encodings) in our new environment with the domain shifts, with the expectation that it would fail due to the large domain shift.

5.2 Training Results

We leverage the training metrics provided by Dennis et al. [2020] in their public code. Some of the most relevant training metrics are shown in Figure 2: "number of objects placed" and "distance to goal" give an estimate for how complicated the environments generated by the adversary are, while "solved path length" can be seen as an estimate of the complexity of the agent's learned behavior. We first trained the unmodified PAIRED agent on the unmodified gym environment and found that the training metrics of our agent roughly agreed (within error) to the metrics shown in Dennis et al. [2020]. Unfortunately, these performance metrics failed to capture underlying issues that were propagated until quite late in the course of this study.

After having trained a large number of each of the static, transfer, and random agents as described in Section 5.1, we compare the training metrics of our new dynamic and static agents and found that they seemed to be performing similarly to the PAIRED baseline (Figure 2), suggesting our approach may be working. Next, we collected performance statistics for the domain transfer experiments. When evaluating the performance of the trained agents in minigrid environments, we found that all of the agents trained in our modified gym environment performed significantly worse than the PAIRED baseline agent we trained. This is concerning, because even our source agent trained on the source environment (i.e. no domain shifts being applied at all) should obtain the same performance as the PAIRED baseline running on the original unmodified environment. There is no possibility of confusion between object types in the environment; at no point is any observation value shared between two different `"WALL", "FLOOR", or "GOAL"` object types. Despite the seeming equivalence of these two observations spaces in both setup and training time, the agents showed terrible behaviour.

We consider two possible reasons for this discrepancy between our modified environment and the original environment. The first is that our new encoding for the black tiles is 0; for the source

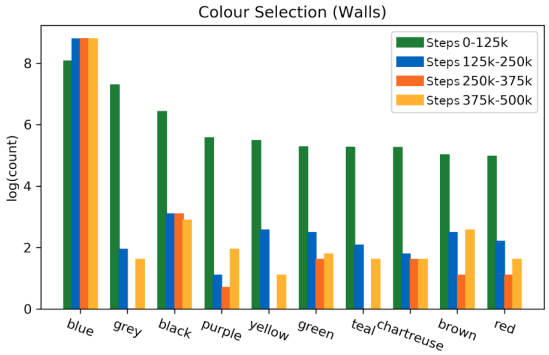


Figure 3: Mode collapse: "blue" is the dominant selection for `wall` objects as early as the first quarter of training. The order is sorted by the number of counts in the first quarter of training for.

environment, this means that the floors are represented by `(0, 0, 0)`. Zero inputs can make it difficult to get good gradients during training (hence why we often use them as padding). To test this theory, we revised our encoding by adding 1 to the first two elements of each 3-tuple (as described in Section 4, the third element is always zero). However, after retraining both a static and dynamic agent with this new encoding, we noticed no improvement in performance.

The second possibility is that by only providing a single value to two entries in the input tuple, we are essentially removing one dimension from the input space. Higher dimensional representations can play better with optimization algorithms, so we naively investigate this by applying a bijective mapping from the input colour encoding to a random number in the same set of values. This generates a unique second number to pair with the first value. We partially trained another static and dynamic agent and we found that performance actually deteriorated significantly. Due to time and compute constraints, this phenomena is not investigated further but is an interesting result that would warrant further investigation.

5.2.1 Mode Collapse

For our dynamic agents, we log what colours they assign to each object type. In all cases, we find that the adversary's selections collapsed to selecting single (arbitrary) set of colours. Figure 3 shows the distribution of selected colours for the `WALL` object type, for an agent that collapsed to selecting `(wall=blue, goal=purple, floor=yellow)` quite early on. After the first quarter of training, the other colour selections made by the agent appear like noise compared to the collapsed colour.

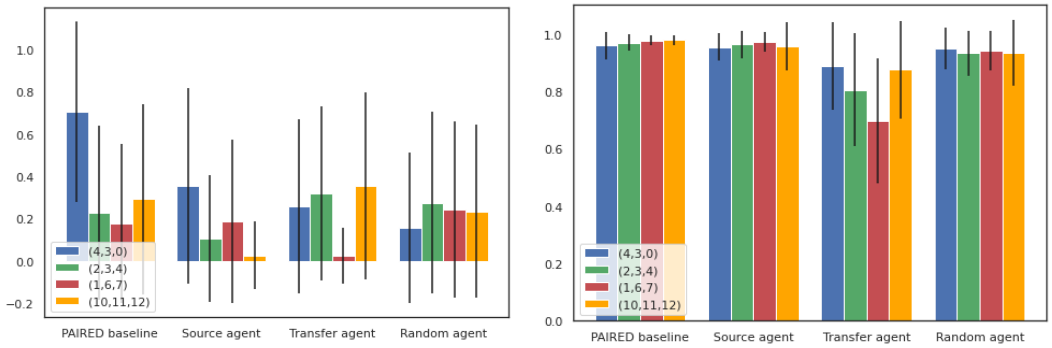
A likely explanation for this behaviour is that the regret (the training signal that the adversarial environment sees) is expressed as the difference in performance between the antagonist and protagonist (equation 1). A new domain shift will likely impact both agents equally, and since this is a difference, its contribution would be cancelled out A simple remedy would be to add a small reward signal to the adversarial environment for selecting a different domain shift.

Another explanation could be that if both agents have a hard time understanding the environment, even without taking the reward function into account there is no reason for the adversary to vary the domain, which would make it even harder for both the agents.

5.3 Zero-shot Task and Domain Transfer Experiments

We now evaluate the performance of the agents discussed above on a set of predefined tasks while also varying the domain. Because the agent performance did not match the baseline cited in Dennis et al. [2020], we restrict our environments to smaller 7×7 grids, which are outlined in Appendix A. The domains that we used are:

- `(4,3,0)`: an arbitrarily picked source environment
- `(2,3,4)`: a target domain that the adversarial environment collapsed to
- `(1,6,7)`: values that are neither part of the mode collapse's domain nor the source domain
- `(10,11,12)`: values that did not exist while the agent was trained (they were not available to the adversary)



(a) (b)

Figure 4: (a) Mean reward in the evaluated environments. (b) Mean reward among successes (i.e. no zero rewards) in evaluated environments. (both) Error bars represent 1σ.

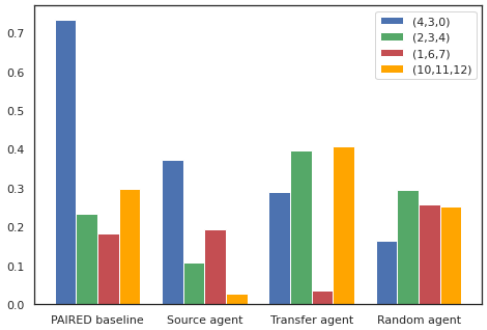


Figure 5: Percentage of evaluation environments solved.

recalling that these tuples represent the integer encodings for a color, corresponding to the environment objects `(wall, goal, floor)` respectively. To reiterate: the source environment is `(4,3,0)`, and all three others are target environments. There is one special case: the agent "PAIRED baseline" did not run on `(4,3,0)`, but instead on the original unmodified gym-minigrid encodings. This is to showcase how much better their agent did on its source environment while still offering competitive generalization performance to our new encodings, all of which are completely novel to the agent.

We evaluate one agent from each regime described in Section 5.1 on each domain and task that we specified for 1000 evaluation episodes. Each episode runs until either the agent completed the task (stepped on the goal), or the agent took 250 steps, in which case this episode is considered a failure. If successful, the environment returns the reward defined in equation (2). We then compute statistics on the task success rates and rewards received.

We first consider the mean total reward among all evaluation episodes, as shown in Figure 4a. Even if we could not reproduce the amazing performance that PAIRED's baseline agent obtained, imPAIRED still produced agents that *seem* to be more robust than both our source agent and the random agent. An emphasis on *seem* is required: the variance of the rewards is huge. This is due to the fact fact that the agent is only able to solve the environment ~30% of the time, where successes had rewards near 1 and failures had rewards near 0. We explore this by only considering the reward obtained by agents which successfully completed the task (see Figure 4b). We find that although the uncertainty decreases considerably, we are still unable to derive statistically significant conclusions about imPAIRED producing agents that better generalize to unseen domains.

We now consider the percentage of solved environments, shown in Figure 5. The source agent's best performance was in its source domain, but had poor performance in the unseen domains. The transfer agent performed relatively consistently on both the source domain and the target domains, and its performance on the target domains was better than the source agent's performance on the same. The random agent performed worse than the source agent on the source agent's domain, but better than

the source agent on the target domains. However, as discussed above, the large uncertainty of rewards indicate that the results found from the present experiments are inconclusive.

6 Conclusions and Future Work

While trends in our domain transfer experiments are promising, more work is necessary in order to make further concrete conclusions. The percentages of solved environments (Figure 5) closely follow our expected results (Section 5.1). This result absolutely isn't probative, but it is still encouraging.

A primary goal of future work will be finding new encodings that allow us to comprehensively reproduce PAIRED's performance before exploring further experiments that builds upon it. Other directions include: Using the same encoding as PAIRED, but shift both the colour and the object type values, using the same encoding as PAIRED, and shifting only the colour value, and instead of using a matrix of 3-tuples, using a single-channel matrix of integers.

Once the encodings are truly equivalent to PAIRED's, there are a few other avenues of interest. We could modify the regret signal to incorporate some signal from the domain shift, hopefully guiding the adversary better in its task. We could also try to separate the environment designer from the domain shifter by introducing a fourth agent whose sole job is to shift the environment after it has been designed. This would allow the adversary to work only in a static domain, without having to deal with domain shifts to its own observations. Lastly, we would explore training agents on multiple domain shifts applied to identical environments at each time step, rather than only picking one shift per step.

References

Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, page 41–48, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161. URL https://doi.org/10.1145/1553374.1553380.

M. Chevalier-Boisvert, L. Willems, and S. Pal. Minimalistic gridworld environment for openai gym. https://github.com/maximecb/gym-minigrid, 2018.

M. Dennis, N. Jaques, E. Vinitsky, A. Bayen, S. Russell, A. Critch, and S. Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. *arXiv preprint arXiv:2012.02096*, 2020.

T. Jaakkola, S. Singh, and M. Jordan. Reinforcement learning algorithm for partially observable markov decision problems. *Advances in Neural Information Processing Systems*, 7, 11 1999.

S. James, P. Wohlhart, M. Kalakrishnan, D. Kalashnikov, A. Irpan, J. Ibarz, S. Levine, R. Hadsell, and K. Bousmalis. Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks. *CoRR*, abs/1812.07252, 2018. URL http://arxiv.org/abs/1812.07252.

B. Mehta, M. Diaz, F. Golemo, C. J. Pal, and L. Paull. Active domain randomization. *CoRR*, abs/1904.04762, 2019. URL http://arxiv.org/abs/1904.04762.

L. J. Savage. The theory of statistical decision. *Journal of the American Statistical Association*, 46 (253):55–67, 1951.

S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, and R. Fergus. Intrinsic motivation and automatic curricula via asymmetric self-play. *arXiv preprint arXiv:1703.05407*, 2017.

S. Sukhbaatar, E. Denton, A. Szlam, and R. Fergus. Learning goal embeddings via self-play for hierarchical reinforcement learning. *arXiv preprint arXiv:1811.09083*, 2018.

J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *CoRR*, abs/1703.06907, 2017. URL http://arxiv.org/abs/1703.06907.

Appendix

A Zero-shot transfer environments

Below is the list of "mini" minigrid environments. These are smaller than the (15×15) grids that the agent was trained on.

- `"MultiGrid-MiniTwoRooms-Minigrid-v0"`
- `"MultiGrid-Empty-Random-6x6-Minigrid-v0"`
- `"MultiGrid-MiniCluttered6-Minigrid-v0"`
- `"MultiGrid-MiniCluttered-Lava-Minigrid-v0"`
- `"MultiGrid-MiniMaze-v0"`
- `"MultiGrid-MiniFourRooms-Minigrid-v0"`
- `"MultiGrid-MiniCluttered7-Minigrid-v0"`
- `"MultiGrid-MiniCluttered1-Minigrid-v0"`

`<num>Rooms` environments feature a number of rooms that separate the grid evenly. `Cluttered` environments are environments filled with many blocks, which is similar to Random environments. Finally, `Maze` environments have the general shape of a maze, forcing the agent to explore the environment fully (since the agent has a limited field of view, it must visit all corridors until it finds the goal).

B Encodings

Table 1: Encoding equivalences

	Minigrid's encoding	Our encoding
Floor	(1,0,0)	(0,0,0)
Wall	(2,5,0)	(4,4,0)
Goal	(8,1,0)	(3,3,0)

C Code

All of the code used to carry out the experiments of the present study, which is an extension publicly released code for PAIRED, is available here: https://github.com/GT-melee/google-research. The provided code did not have a usable evaluation script, as it was tied to the internal Google compute infrastructure. As such, we developed our own script used to evaluate the domain transfer experiments, which is available here: https://github.com/GT-melee/google-research/blob/master/eval.py.